

AP CS A Exam Exam Overview

Section I: Multiple Choice

42 Questions | 1 Hour 30 Minutes | 55% of Exam Score

- The multiple-choice section includes mostly individual questions, occasionally with 1–2 sets of questions (2 questions per set).
- Computational Thinking Practices 1, 2, 4, and 5 are all assessed in the multiple-choice section.

No penalty for guessing!

Section II: Free Response

4 Questions | 1 Hour 30 Minutes | 45% of Exam Score

- All free-response questions assess Computational Thinking Practice 3: Code Implementation, with the following focus
 - **Question 1:** Methods and Control Structures—Students will be asked to write program code to create objects of a class and call methods, and satisfy method specifications using expressions, conditional statements, and iterative statements.
 - **Question 2:** Class Design—Students will be asked to write program code to define a new type by creating a class and satisfy method specifications using expressions, conditional statements, and iterative statements.

- **Question 3:** Data Analysis with ArrayList—Students will be asked to write program code to satisfy method specifications using expressions, conditional statements, and iterative statements and create, traverse, and manipulate elements in ArrayList objects.
- **Question 4:** 2D Array—Students will be asked to write program code to satisfy method specifications using expressions, conditional statements, and iterative statements and create, traverse, and manipulate elements in 2D array objects.

Even though not explicitly listed, Strings are very popular on the Free Response part of the AP exams. Know the string methods listed on the reference sheet really well!

[Java Reference Sheet](#)

or here:

<https://apcentral.collegeboard.org/pdf/ap-computer-science-a-java-quick-reference.pdf>

What's New in the 2025–26 Course

The course is now 4 units instead of 10. Four things changed that affect this exam:

GONE: inheritance and polymorphism. No extends, super, method overriding, or the Object class. Overriding toString is also out of scope.

NEW: reading text files. Know File, Scanner(File f), hasNext(), close() and String.split(). Reading only — there is no file writing on the exam.

NEW: data sets and the ethics of collecting them — privacy risk, data quality, and choosing an appropriate data set for a problem. **Also watch out: the wrapper constructors new Integer(5) and intValue() are off the reference sheet. Use Integer.parseInt and Integer.MIN_VALUE / MAX_VALUE instead.**

General Scoring

MCQ Score: $x / 42$ (55% of exam score)

FRQ Score: $y / 25$ (45% of exam score)

Composite: $0.55(x / 42) + 0.45(y / 25)$

AP Score (approximate — based on past curves):

5 composite $\geq 78\%$

4 composite $\geq 59\%$

3 composite $\geq 46\%$

Multiple Choice

Goal: Everyone should be able to get at least 21 out of 42 questions correct.

During the first pass through the exam, only spend time on “easy” problems: problems are short and succinct and you know you can get right if you are careful. **SKIP** hard or wordy questions that ask about sorting algorithms(merge, selection, insertion) or binary search.

During the second pass, spend time on the “medium” questions; questions that you are confident that you can get right but might need a bit more time.

Note: For some, strategically it might make sense to **entirely skip 4-8 of the hardest MC questions and instead spend more time on the other questions. Guess the answers to these skipped questions at the end. Each question now has 4 answer choices (down from 5), so a blind guess is 1 in 4.**

Free Response

General Strategies for FRQS:

- 1) Read the questions before you read the code! This will help you focus.
- 2) Unless a question specifically addresses efficiency, focus on clear, simple code rather than complicated, efficient code.
- 3) Comments are not necessary. Use only to organize your thoughts.
- 4) Pay attention to the code they give you:
 - Do you have more than one class? Use it!
 - Did they give you methods? You will need them!
 - Are there instance variables that you should use? If so, use them. Do not re-declare.
 - Only use accessor and mutator methods to access private instance variables.
 - Do not re-write or change method headers.
 - Use parameters (names and types) as given

Free Response

General Strategies for FRQS:

5) If part (b) references a method written in part (a), you will likely need to use this method. If it is not clear to you how, stop and think before you write.

6) Pay attention to the return type. If it is not void, be sure to include a return statement!

7) If a method comment says `/* Implementation not shown */`. You do not to implement the method. But you'll likely need to use the method in your code at least once.

Free Response

General Strategies for FRQS:

8) Write down some code for every question! **Leave Nothing Blank! Get partial credit if not full credit!**

- does it need a loop?
- an if statement?
- return statement or assignment statement?

Example 1:

```
public double funMethod(){  
    double result;  
    ...  
    return result;  
}
```

More Examples

Leave Nothing Blank! Get partial credit if not full credit!

```
public int[] funMethod(int length){  
    int[] list = new int[length];  
    ...  
    return list;  
}  
  
public ArrayList<Bug> funMethod(){  
    ArrayList<Bug> buggies = new ArrayList<Bug>();  
    ...  
    return buggies;  
}
```

More Examples

Leave Nothing Blank! Get partial credit if not full credit!

```
public boolean funMethod(int something){  
    ...  
    if(something > somethingElse)  
        return true;  
    // must have a default return; every return can't be  
    // locked in an if  
    return false;  
}
```

Question 1: Methods and Control Structures

See previous years' similar FRQs:

2018 #1, 2019 #1, 2017 #3, 2018 #2, 2021#1

Methods and Control Structures

Question 1 of the FRQ will likely ask you to write methods that utilize control structures(for vs while loops, conditionals).

You are given (usually static) methods with parameters and will be asked to write a method that calls one of the given methods in its implementation.

If a method comment says `/* Implementation not shown */`. You do not to implement the method. But you'll likely need to use the method in your code at least once.

AVOID using arrays/arraylists for this problem.

2019 #1 is a good example of this problem.

Methods and Control Structures

Likely you will need to use for loops and if-else if-else conditionals.

Likely your method will return some value. See its return type in its header.

Using Strings and its methods can be required here in this question(or Question 2). See for example 2017 #3 for a String question that fits "Methods and Control Structures" question.

You should know how to use **substring(int beg, int end)**, **substring(int beg)**, **indexOf**, **equals**, **length** and **compareTo** methods. See the College Board reference sheet.

String methods

Method name	Description
<code>String(String str)</code>	Constructs a new String object that represents the same sequence of characters as <i>str</i>
<code>int length()</code>	Returns number of characters in this string
<code>substring(index1, index2)</code> or <code>substring(index1)</code>	Returns the characters in this string from <i>index1</i> (inclusive) to <i>index2</i> (<u>exclusive</u>); if <i>index2</i> is omitted, grabs till end of string
<code>boolean equals(Object other)</code>	Returns true if this is equal to other; returns false otherwise
<code>int compareTo(String other)</code>	Returns a value < 0 if this is less than other; returns zero if this is equal to other; returns a value > 0 if this is greater than other
<code>indexOf(str)</code>	Returns index where the start of the given string appears in this string (-1 if not found)
<code>String[] split(String del)</code>	Returns an array of the substrings of this string, split around <i>del</i>

String method examples

```
// index      0123456789012345678
String s1 = "programming in java";
System.out.println(s1.length());
// 19
System.out.println(s1.indexOf("i")); // 8
System.out.println(s1.indexOf("gram")); // 3
System.out.println(s1.indexOf("hi")); // -1

System.out.println(s1.substring(7, 10)); // "min"
System.out.println(s1.substring(12)); // "in java"
System.out.println(s1.substring(2,3)); // "o"

String s2 = s1.substring(10, 17); // "g in ja"
```

indexOf Example

indexOf can be confusing. Here's an example.

```
public static mystery(String str){
    String answer = "";
    for(int i = 0; i < str.length(); i++){
        String currentLetter = str.substring(i,
i+1);
        if("aeiou".indexOf(currentLetter) != -1)
            answer += current;
    }
    return answer;
}
```

What is the output?

```
System.out.println(mystery("cat in hat"));
```

Answer: aia

Do Problem 2018 #2

Complete 2018 #2.

This is a good problem that uses 1D array, ArrayList, and Strings and is a good Question 1 (Methods/Control Structures) type of problem.

Also do the following 2018 #1, 2019 #1, 2017 #3, 2021 #1 to prepare for Question 1.

Question 2: Class Design

See previous years' similar FRQs:

2019 #2, 2021 #2, 2016 #1, 2015 #2, 2021#2

Question 2

This question will ask you to write an entire class including private instance variables, constructors and methods.

The class implementation typically involves 2 – 4 instance variables, one constructor and 2 – 3 instance methods.

AVOID using arrays/arraylists for this problem. Otherwise you'll be doing needless for loops. Consider a variable(int or double) that accumulates(sums) values rather than keeping track of each value in an array/arraylist.

The Point class' implementation on the next slide provides an example of the anatomy of a class.

An Implementation of Point

```
public class Point {  
    private int x;  
    private int y;  
    public Point(int newX, int newY){  
        x = newX;  
        y = newY;  
    }  
    public void translate(int dx, int dy){  
        x += dx;  
        y += dy;  
    }  
    public double distanceToOrigin(){  
        return Math.sqrt(x * x + y * y);  
    }  
}
```

instance variables

constructors to initialize instance variables.

mutator method

accessor method

Typically, the question will give you examples of method calls and corresponding outputs and require you to create a class that satisfies those specifications. 2019 # 2 is a great example of this.

explains you how to
write your constructor!

Statements and Expressions	Value Returned (blank if no value)	Comment
<code>StepTracker tr = new StepTracker(10000);</code>		Days with at least 10,000 steps are considered active. Assume that the parameter is positive.
<code>tr.activeDays();</code>	0	No data have been recorded yet.
<code>tr.averageSteps();</code>	0.0	When no step data have been recorded, the <code>averageSteps</code> method returns 0.0.
<code>tr.addDailySteps(9000);</code>		This is too few steps for the day to be considered active.
<code>tr.addDailySteps(5000);</code>		This is too few steps for the day to be considered active.
<code>tr.activeDays();</code>	0	No day had at least 10,000 steps.
<code>tr.averageSteps();</code>	7000.0	The average number of steps per day is (14000 / 2).

And what methods to
implement!

StepTracker

Students have implemented correctly StepTracker(2019 #2) using an ArrayList. **However, avoid using ArrayList/1D Arrays or 2D arrays for #2.**

Otherwise, you'll be doing needless for loops in your implementation.

The previous problem(2019 #2) is simple problem that involves only working with one class.

Another problem that is slightly more complex is 2021 #2. This problem involves writing a class that uses ANOTHER given class.

In this problem, you are given a SingleTable class with accessors and mutators methods and you are asked to write the CombinedTable class that uses the SingleTable class.

Try this problem. Here's a hint: A subtle error to this problem is not using SingleTable instance variables! If you use int and double instance variables, your answer will not receive full credit beca

<code>t2.setViewQuality(80);</code>		Changing the view quality of one of the tables that makes up <code>c2</code> changes the desirability of <code>c2</code> , as illustrated in the next line of the chart. Since <code>setViewQuality</code> is a <code>SingleTable</code> method, you do not need to write it.
<code>c2.getDesirability();</code>	67.5	Because the view quality of <code>t2</code> changed, the desirability of <code>c2</code> has also changed.

Question 2

Thus, the take away is that if the problem gives you a class(SingleTable) and requires you to use it another class(CombinedTable), it is likely that you will need to **make at least one instance variable(for example, SingleTable t1) in the class that you are writing(CombinedTable).**

Please do the following 2019 #2, 2021 #2, 2020 #2, 2016 #1, 2015 #2 and check your solutions to prepare for Question 2.

Question 3: Data Analysis with ArrayList

See previous years' similar FRQs:

2010 #1, 2013 #1, 2017 #1, 2018 #2, 2021#3

ArrayList — Question 3

We will first prepare for the FRQs by reviewing ArrayList and 1D arrays. Question 3 is now Data Analysis with ArrayList.

As of the 2025-26 revision, Question 3 is strictly an ArrayList question — the College Board removed 1D arrays from it. Arrays are still on the exam elsewhere (Question 1 and the multiple choice), so review both, but expect only ArrayList in Question 3.

See the following problems 2010 #1, 2013 #1, 2017 #1, 2018 #2

It's very common to have a class(e.g Student) and another class which contains either a 1D array of Student objects or an ArrayList of Student objects.

```
public class Student {  
    ...  
    public String getName(){..}  
    public double getGpa(){...}  
}
```

ArrayList/1D Arrays

```
public class Course {  
    private ArrayList<Student> students;  
  
    public Course() {  
        students = new ArrayList<Student>();  
        ...  
    }  
}
```

OR

```
public class Course {  
    private Student[] students;  
  
}  
}
```

Traversing 1D Array vs. ArrayList

Arrays:

```
double sum = 0;
for(int i = 0; i < students.length; i++){
    sum += students[i].getGpa();
}
```

ArrayLists:

```
double sum = 0;
for(int i = 0; i < students.size(); i++){
    sum += students.get(i).getGpa();
}
```

Traversing 1D Array vs. ArrayList

Use for each loops if you are only traversing the arraylist and **not removing items from it.**

Both Arrays and ArrayLists:

```
double sum =0;
for(Student s: students){
    sum += s.getGpa();
    // notice no [i] and no get(i)!!
}
```

ArrayList

Remember for ArrayLists that if you remove an item, items to the right of it will shift left!

And if you add an item to the list, items will shift right to accommodate!

Make sure to take this into account when you are adding or removing items.

ArrayList/1D Array Tips

When using the ArrayList method `remove()`, remember you can only REMOVE BY INDEX (not by object), so you must use a traditional for loop (and NOT a for each loop).

```
for(int i=0; i<list.size(); i++){  
    if(list.get(i).getScore() <= 50)  
        list.remove(i);  
    i--; //AND don't forget to decrement the index!  
}
```

//OR traverse the list backwards, i.e.

```
for(int i=list.size()-1, i>=0, i--){  
    if(list.get(i).getScore() <= 50)  
        list.remove(i--);  
}
```

ArrayList/1D Array Tips

Avoid array and ArrayList out-of-bounds exceptions! Especially when comparing consecutive elements.

Example: You need to compare consecutive elements to determine if the values are increasing.

```
public boolean increasing(int[] numbers){  
    for(int i=0; i < numbers.length-1; i++)  
        if(numbers[i] >= numbers[i+1])  
            return false;  
    return true;  
}
```

Finding the smallest or largest

When you need to return an element in a list (i.e. largest, smallest,...), assign the initial element to the **first one** in the list.

Example : You need to find the Dog with the most fleas.

//returns the Dog object in pack with the most fleas

```
public Dog mostFleas(ArrayList<Dog> pack){  
    Dog most = pack.get(0);  
    for(Dog dog : pack)  
        if(dog.getNumFleas() > most.getNumFleas())  
            most = dog;  
    return most;  
}
```

Manipulating an Array using ArrayList

An ArrayList can be helpful when you are trying to manipulate an array (i.e. remove items, rearrange in random order, etc.).

Example:

If you need to remove several objects from an array based on some condition, a strategy would be to save all the objects to an ArrayList and remove the objects before storing the remaining objects back to the array.

See code on the next slide.

Manipulating an Array using ArrayList

```
//returns a new array with bad things removed
public Thing[] removeBadThing(Thing[] widgets){
    ArrayList<Thing> good = new ArrayList<Thing>();
    for(Thing wid : widgets){
        if(!wid.isBad()) // i.e good
            good.add(wid);
    }
    Thing[] goodOnes = new Thing[good.size()];
    for(int i=0; i < goodOnes.length; i++)
        goodOnes[i] = good.get(i);
    return goodOnes;
}
```

Question 3

Please do the following 2010 #1, 2013 #1, 2017 #1, 2018 #2, 2021 #3 and check your solutions to prepare for Question 3.

Question 4: 2D Array

See previous years' similar FRQs:

2019 #4, 2018 #4, 2017 #4, 2016 #3, 2021#4

Row Major Order Traversal

Example : Count the number of Students objects whose GPA is above a threshold. (row major order)

```
public class Course{
    Student[][] students;
    // constructors not shown
    public int aboveThreshold(double threshold){
        int count = 0;
        for(int row = 0; row < students.length; row++){
            for(int col = 0; col < students[0].length; col++){
                if(students[row][col].getGpa() > threshold)
                    count++;
            }
        }
        return count;
    }
}
```

Column Major Order Traversal

Example : Count the number of Students objects whose GPA is above a threshold. (column major order)

```
public class Course{
    Student[][] students;
    // constructors not shown
    public int aboveThreshold(double threshold){
        int count = 0;
        for(int col = 0; col < students[0].length; col++){
            for(int row = 0; row < students.length; row++){
                if(students[row][col].getGpa() > threshold)
                    count++;
            }
        }
        return count;
    }
}
```

Along One Row

Example : Count the number of Students objects whose GPA is above a threshold in the given **row**.

```
public class Course{
    Student[][] students;
    // constructors not shown

    public int aboveThreshold(int row, double threshold){
        int count = 0;
        for(int col = 0; col < students[row].length; col++){
            if(students[row][col].getGpa() > threshold)
                count++;
        }
        return count;
    }
}
```

Along One Column

Example : Count the number of Students objects whose GPA is above a threshold in the given **column**.

```
public class Course{
    Student[][] students;
    // constructors not shown
    public int aboveThreshold(int col, double threshold){
        int count = 0;
        for(int row = 0; row < students.length; row++){
            if(students[row][col].getGpa() > threshold)
                count++;
        }
        return count;
    }
}
```

Major Diagonal Traversal

Example : Count the number of Students objects whose GPA is above threshold along the **major diagonal**. Assume 2D array is square.

```
public class Course{
    Student[][] students;
    // constructors not shown
    public int aboveThreshold(double threshold){
        int count = 0;
        for(int i = 0; i < students.length; i++){
            if(students[i][i].getGpa() > threshold)
                count++;
        }
        return count;
    }
}
```

Minor Diagonal Traversal

Example : Count the number of Students objects whose GPA is above threshold along the **minor diagonal**. Assume 2D array is square.

```
public class Course{
    Student[][] students;
    // constructors not shown
    public int aboveThreshold(double threshold){
        int count = 0;
        int dim = students.length;
        for(int i = 0; i < dim; i++){
            if(students[i][dim - 1 - i].getGpa() > threshold)
                count++;
        }
        return count;
    }
}
```

Question 4

Please do the following 2019 #4, 2018 #4, 2017 #4, 2016 #3, 2021 #4 and check your solutions to prepare for Question 4.